

Bloom v2 roadmap

The approved blueprint turned into seven small phases, each with concrete tasks, dependencies and testable acceptance criteria. Along the way, "Learn" boxes explain how the system and the codebase are organised and why each step is built the way it is.

Source [docs/bloom-coach-blueprint](#) Baseline [docs/bloom-as-built](#) Written **5 Oct 2026**

Size: **S** ≈ 1 day · **M** ≈ 2–3 days · **L** ≈ 1 week, for one developer. These are rough, for ordering only.

00 Overview

Each phase ships something usable on its own, behind the existing per-cohort feature flag, so one pilot group can try it first. Old paths keep working the whole time.

PHASE	OUTCOME FOR STUDENTS AND MENTORS	VISIBLE CHANGE	SIZE
0 · Groundwork	Decisions recorded; AI calls logged and limited; the known Bloom bugs fixed; a test runner in place	Bug fixes only	~1.5 wk
1 · Activities	Paths made of interactive activities (lesson cards, quick checks, short answers, decisions, reflections) instead of tick boxes	Yes, in the pilot cohort	~3 wk
2 · Feedback	Written answers get instant, rubric-based feedback and can be improved	Yes	~2 wk
3 · Evidence & artifacts	Interviews, templates and photo or file uploads; accepted work becomes a versioned project artifact	Yes	~3 wk
4 · Project memory	Bloom remembers confirmed project facts and recommends what's missing for the current stage	Yes	~2 wk
5 · Coach & mentors	Ask Bloom becomes a conversation with memory; mentors get a review queue and the problem-statement checkpoint	Yes. MVP complete.	~2.5 wk
6 · Clean-up	Old columns and compatibility code removed after full rollout	No	~0.5 wk

Phase 5's Ask Bloom work only needs phases 0 and 2, so a second developer can build it alongside phases 3 and 4 (see [dependencies](#)).

01 Before we start: how the repo and the organizations are set up

LEARN · REPOSITORY

Two products live in one repository

This repo began as a fork of an open-source AI tutor called *Bloom* (`upstream` remote, Li-Evan/Bloom). Youth Idea Lab was then built beside it. What you'll find:

FOLDER	WHAT IT IS	STATUS
<code>apps/web/</code>	The Youth Idea Lab product: Next.js 16, React 19, server components and server actions; student, parent, mentor and admin views. Bloom v2 is built here.	Active
<code>supabase/</code>	The database as code: <code>migrations/</code> (schema, RLS, triggers), <code>seed.sql</code> (the 8-week programme), <code>tests/</code> (pgTAP permission tests).	Active
<code>root</code> <code>package.json</code>	Database scripts: <code>db:push</code> , <code>db:test</code> , <code>db:types</code> .	Active
<code>backend/</code> , <code>frontend/</code> , <code>skills/</code> , <code>example/</code> , <code>Makefile</code>	The original Bloom tutor (FastAPI + SQLite + React/Vite), with Chinese sample courses.	Legacy. The README says this lives on <code>bloom/legacy</code> , yet the files are still tracked on this branch.
<code>site/</code>	An Astro website.	Separate, not part of this plan
<code>docs/ARCHITECTURE.md</code>	Describes the <i>legacy</i> tutor, not <code>apps/web</code> .	Misleading; task 0.5 fixes it

Why it matters: the old tutor already has ideas we want, such as recommendations and a learning profile. But copying from it, or reading its architecture doc by mistake, will confuse new contributors. Phase 0 labels it clearly.

LEARN · DATA MODEL

Organizations, programmes, cohorts and roles

The schema separates the **template** (what the course is) from the **instance** (a group of real people taking it). The first migration says so in its header comment.

<code>organizations</code>	who runs the course (one row today: Youth Idea Lab)
├ <code>programs</code> (template)	"8-week course", weeks = 8
└ <code>stages</code>	Explore 1-2 · Choose 3-4 · Build 5-6 · Present 7-8
└ <code>activities</code>	weekly tasks, per age group (null = both)

```

└─ cohorts      (instance)      "Group 1", start_date, timezone, schedule
    └─ memberships      user + cohort_role (student | mentor) + age_group
    └─ sessions          live classes
    └─ per student: projects (1 per cohort) · submissions → feedback · progress_cards ·
bloom_paths

profiles.role (app_role)      student | parent | mentor | admin    (one global role per account)
guardian_links                parent ↔ student
consents                      parent grants per student: platform, ai, bloom_ai, media,
public_portfolio

```

How access works. Every table has row-level security (RLS): Postgres itself filters rows for the signed-in user. Small helper functions answer one question each, and policies combine them:

- `is_admin()` : the account's role is admin.
- `is_cohort_member(c)` / `is_cohort_mentor(c)` : the user belongs to, or mentors, cohort c.
- `is_parent_of(s)` : there is a guardian link.
- `mentors_student(s)` : the user mentors a cohort that student s is in.
- `can_view_student(s)` : self, or parent, or mentor, or admin. Bloom uses this one.

The organization caveat. `organization_id` is stored on profiles, programs, cohorts and invitations, but **no policy uses it**. `is_admin()` doesn't check the organization, so an admin can see every organization's data, and any signed-in user can read the organizations list. In practice the platform is *single-tenant*. That's fine for one organization; before a second partner organization is onboarded, admin rights must be scoped to an organization. Bloom v2 doesn't depend on this. New tables reach the organization through the student, so they need no `organization_id` of their own.

LEARN · LAYERS

Who enforces what

LAYER	IN THIS CODEBASE	JOB	BLOOM V2 USES IT FOR
Server components	<code>app/**/page.tsx</code>	Read data and render HTML on the server	Path, activity and mentor pages
Client components	<code>"use client"</code> forms	Interaction in the browser	Activity players, feedback, threads
Server actions	<code>actions.ts</code> (<code>"use server"</code>)	Validate input, run the business rules, call services	<code>submitActivity</code> , <code>confirmFact</code> , ...
Domain modules	<code>lib/</code> (new <code>lib/bloom/</code>)	Pure rules that are easy to unit test	Catalogue, state machine, verdict mapping, brief
RLS policies	migrations	The last line of defence: who can read and write which rows	Drafts hidden from parents, ownership
Triggers	migrations	Rules that must hold no matter who writes	Copying <code>student_id</code> , timestamps, path status

LAYER	IN THIS CODEBASE	JOB	BLOOM V2 USES IT FOR
Security-definer RPCs	<code>set_bloom_mentor_note</code>	Privileged operations with their own checks	Mentor review, atomic path creation

Rule of thumb: **the UI is for convenience, actions for business rules, the database for guarantees**. If something must never happen (a parent reading a draft), the database blocks it, not just the UI.

P0 Phase 0: Groundwork

Goal **make the next phases safe to build** Flag **none** Depends on **nothing**

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
0.1	Decision record <code>docs/decisions/0001-bloom-v2.md</code> , settling the blueprint's open questions with defaults: parents keep full Ask Bloom visibility (matches the current consent); photos allowed with guidance and location data removed; English only for the MVP; course activities stay separate; model per AI capability.	—	S	- Each decision has an owner, a date and a reason - Product and safeguarding leads have signed off
0.2	Unit test runner: add Vitest to <code>apps/web</code> with an <code>npm test</code> script.	—	S	<code>npm test</code> runs and passes, including one sample test of <code>lib/programme.ts</code> <code>currentWeek</code>
0.3	AI gateway: move <code>lib/ai.ts</code> to <code>lib/ai/</code> (<code>gateway.ts</code> plus one file per capability). Add an <code>ai_runs</code> table (capability, student, model, tokens in and out, latency, outcome, input hash) written with the server key. Add a daily call limit per student.	0.2	M	- Suggest, plan, ask and the progress card behave exactly as before - Every call writes exactly one <code>ai_runs</code> row; no prompt text is stored - Going over the daily limit returns the existing "rate limited" message - Only admins can read <code>ai_runs</code> (pgTAP)

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
0.4	Fix the as-built defects: a <code>create_bloom_path</code> RPC that inserts the path and its tasks in one transaction; a trigger on <code>bloom_tasks</code> that recomputes the path status; restoring an archived path recomputes it; saving no longer wipes the note.	—	M	- pgTAP: a failing task insert leaves no path behind - pgTAP: adding a task to a completed path makes it active - pgTAP: restoring an all-done path makes it completed - Manual: reopen a task with a note, then Mark done; the note is kept
0.5	Docs: rename <code>docs/ARCHITECTURE.md</code> to <code>docs/legacy-tutor-architecture.md</code> , add <code>docs/architecture.md</code> for <code>apps/web</code> (a one-page map plus links to as-built, blueprint and roadmap), and decide whether to remove the legacy folders from this branch.	—	S	A new contributor following the README reaches the right architecture page
0.6	Rollout flag: use the existing <code>isEnabled("bloom_v2", cohortId)</code> from <code>lib/flags.ts</code> (built but unused today) in the Bloom pages and actions.	—	S	- Flag off: the UI is identical to today - Flag on for one cohort: only that cohort sees v2 entry points

Exit: all pgTAP and unit tests green; AI usage visible per day; decisions signed off.

LEARN

Why a gateway before features

Today each AI function builds its own client and handles errors itself, and nothing is recorded. With a single gateway, consent, limits, model choice and logging happen in one place. Every later capability (assessor, coach, memory proposer) gets them for free, and we can answer "what does Bloom cost per student?" from `ai_runs`. The input *hash* lets us spot repeated calls without storing what children wrote.

P1 Phase 1: Interactive activities

Goal **paths made of activities students actually do** Flag **bloom_v2** (pilot cohort) Depends on **0.2, 0.4, 0.6**

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
1.1	Expand migration on <code>bloom_tasks</code> : <code>type</code> (default <code>legacy</code>), <code>config</code> jsonb, <code>config_version</code> , <code>required</code> , <code>is_milestone</code> , <code>origin</code> , <code>revision_count</code> , and a new <code>state</code> column using the new enum (available ... skipped). Backfill <code>state</code> from <code>status</code> and keep the two in sync with a trigger. Regenerate types.	0.4	M	- Every existing row has a matching state (todo→available, doing→in_progress, done→complete) - The existing UI works unchanged - pgTAP: students can't change <code>type</code>, <code>config</code> or <code>is_milestone</code> (column grants)
1.2	Activity catalogue <code>lib/bloom/activities/</code> : a registry plus <code>learn</code> , <code>quick-check</code> (choice, sort), <code>respond</code> , <code>decide</code> , <code>reflect</code> and <code>legacy</code> . Each defines a config schema (Zod), an evidence spec and a completion rule.	0.2	M	- Unit tests: valid and invalid config samples for every type - A quiz whose answer key points to a missing option is rejected
1.3	State machine <code>lib/bloom/state.ts</code> : a pure <code>transition(activity, event)</code> plus an apply step that updates <code>where</code> <code>state = expected</code> .	1.1, 1.2	M	- A unit test for every edge in the blueprint diagram; illegal moves rejected - Double-clicking Submit produces exactly one submission
1.4	Submissions table <code>bloom_submissions</code> (activity, attempt, content jsonb, status draft or submitted) with RLS: the student reads and writes their own; others read <i>submitted</i> rows only. Add the audit trigger and include the table in <code>export_student_data</code> .	1.1	M	- pgTAP: a parent can't see drafts; a mentor sees submitted work of their own students only; a student can't write to someone else's activity - The data export contains submissions
1.5	Actions <code>saveDraft</code> , <code>submitActivity</code> , <code>reopenActivity</code> , using the catalogue and the state service.	1.3, 1.4	M	- Content is validated on the server against the evidence spec, with errors per field - Instant types complete on submit; the path status updates

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
1.6	Activity players: Learn cards; Quick check with choice and tap-to-sort (no drag, so it works on phones and with a keyboard); Respond; Decide; Reflect. Legacy tasks keep <code>TaskControls</code> .	1.5	L	• Works at 360 px wide and by keyboard alone • Drafts survive a reload • Pending states use <code>SubmitButton</code> • A wrong quick-check answer shows a hint and allows one retry, then completes
1.7	Planner v2: outputs <code>{phase, type, config}</code> as a Zod discriminated union over the catalogue; normalises counts and phase order; one repair retry; saves through <code>create_bloom_path</code> . Behind the flag.	0.3, 1.2	M	• Of 20 generated plans (both age groups, 4 stages), at least 95% pass validation without the retry • A failed plan saves nothing and shows a clear message • Every plan has a Do-phase activity
1.8	Migrate notes: each non-empty <code>bloom_tasks.reflection</code> becomes a submitted <code>bloom_submissions</code> row; the legacy UI reads notes from submissions.	1.4	S	• The number of migrated rows equals the number of non-empty reflections • Notes show as before

Exit: in the pilot cohort, a student completes a 5-activity v2 path on a phone; legacy paths in the same account still work.

LEARN

Expand, migrate, contract

Changing a live schema safely happens in three moves. **Expand:** add new columns and tables beside the old ones, keeping them in sync (1.1). **Migrate:** move the code and the data over gradually (1.5–1.8, and later phases). **Contract:** remove the old parts only once nothing uses them (phase 6). At every point the app works, and a bad deploy can roll back without data loss. Postgres enums are a good example: you can add values but not easily remove them, so the plan adds a new `state` column instead of reshaping `status` .

Why the state machine is a pure function

`transition(current, event) → next` has no database or network inside it, so every rule can be unit tested in milliseconds. The database step then writes with a guard (`where state = 'in_progress'`). If two requests race, only the first matches, which is called *optimistic concurrency*.

P2 Phase 2: The feedback loop

Goal students improve work based on specific feedback Flag `bloom_v2` Depends on phase 1

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
2.1	Assessments table <code>bloom_assessments</code> (submission, by auto, ai or mentor, criteria jsonb, verdict, feedback, <code>ai_run_id</code>). Parents can't read AI assessments (blueprint section 13).	1.4	S	pgTAP: student and mentor read; parent doesn't; nobody updates (rows are append-only)
2.2	Rubrics in the catalogue for <code>respond</code> : criteria with <code>required</code> flags and wording for each age group.	1.2	S	An educator has reviewed every criterion's wording
2.3	Context assembler v1 <code>lib/bloom/context.ts</code> : named slices with token budgets (estimated as characters ÷ 4); replaces <code>bloomContext</code> .	0.3	M	- Unit: no slice exceeds its budget - Unit: a fixture containing the student's name or username never appears in the output
2.4	Assessor and verdict mapping: AI returns met or not yet per criterion; the app decides the state. Any required criterion not yet, with fewer than 2 revisions, means needs revision. Otherwise complete. An AI error means complete, flagged "not reviewed".	2.1– 2.3	M	- Unit tests for the mapping, including the cap and fail-open Eval: 30 sample answers labelled by an educator; at least 80% agreement on required criteria; no marks or percentages in any feedback - p95 submit-to-feedback under 12 s
2.5	Feedback UI: per-criterion list, Resubmit, "Keep as is", attempt history.	2.4	M	- Feedback persists on reload "Keep as is" completes a non-milestone activity and flags it for the mentor

Exit: the pilot shows a revision rate and an accept rate per activity (queried from `bloom_assessments`); no student is ever stuck because of AI.

LEARN

Evals: testing AI like any other function

Unit tests can't tell you whether feedback is *good*. An **eval** is a fixed set of inputs with expected judgements, made by a human expert. You run it whenever a prompt, rubric or model changes, and compare the agreement number. Keep the set in the repo (anonymised and invented, never real student work) and treat a drop in agreement like a failing test.

Fail-open versus fail-closed

When a check can't run, you either allow (fail-open) or block (fail-closed). Bloom's *learning* checks fail open, because a child shouldn't be stuck when an API is down; the mentor sees the "not reviewed" flag. *Safety and permission* checks fail closed: no

consent means no AI call, and a failed PII check means no submission.

P3 Phase 3: Evidence and artifacts

Goal **real-world work captured and kept on the project** Flag **bloom_v2** Depends on **phase 2, decision 0.1**

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
3.1	Consent v2: new <code>bloom_ai</code> consent version covering project work and photos; a parent re-consent prompt; the gateway refuses image input without v2.	0.1, 0.3	M	- A v1-only student can still use text features - Image assessment needs v2 (unit plus manual) - Parent page shows the new wording
3.2	Evidence storage: private bucket <code>bloom-evidence</code> , path <code>{student}/{submission}/{uuid}.{ext}</code> , storage policies (owner writes; <code>can_view_student</code> reads submitted work); a <code>bloom_submission_files</code> table; images resized in the browser to a 1600 px max edge; location (EXIF) data stripped on the server; 10 MB limit.	1.4	L	- A test photo with GPS data is stored without it - A parent can't fetch a draft's file - An 11 MB file is rejected by the server, not just the browser
3.3	PII check <code>lib/bloom/pii.ts</code> : emails, phone numbers (including +92 and 03xx formats) and street-address patterns; blocks the submit with a friendly explanation.	0.2	S	Unit: true positives and a list of known false positives ("5 questions", prices) pass
3.4	Interview type: config, player (question picker that flags closed questions, answer per question, interviewee role, optional photos), rubric.	2.4, 3.2, 3.3	L	- Worked-example step 2 runs end to end - A 1-word answer to a required question gets a "not yet" in the eval
3.5	Make type: templates <code>business_map</code> and <code>problem_statement</code> , plus a generic image or file; images passed to the assessor (vision) for consistency only.	3.1, 3.2, 3.4	L	- Worked-example step 3 runs end to end - Feedback never comments on drawing skill (eval)

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
3.6	Artifacts: <code>artifacts</code> and <code>artifact_versions</code> tables plus a service. An accepted Interview or Make submission creates version n+1 and moves <code>current</code> ; versions can never be edited. Artifact page with the version list.	3.4	M	- pgTAP: no update grant on versions - An improved resubmission creates v2 and v1 stays readable - The parent sees the current version only

Exit: the pilot produces real interview sets and business maps, with photos and no location data.

LEARN

How file security works here

Supabase Storage applies RLS policies to file paths too. The existing course uploads use `<cohort_id>/<student_id>/<file>` (`lib/storage.ts`), and the storage policies parse that path to decide access. Files are never public. The app creates *signed* URLs that expire after 10 minutes. Bloom evidence follows the same pattern with its own bucket, so its rules can differ (drafts stay private).

Evidence versus artifacts

A **submission** answers "did the student do this activity?". An **artifact version** answers "what does the project have now?". Keeping them separate means a quiz never clutters the project, and an artifact can later be fed by course submissions too.

P4 Phase 4: Project memory and recommendations

Goal Bloom builds on what the student has already done Flag `bloom_v2` Depends on 3.6

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
4.1	Facts table <code>project_facts</code> (type, statement, status proposed, confirmed, dismissed or superseded, source, <code>confirmed_by</code> , <code>supersedes_id</code>), plus <code>projects.target_user</code> . Only confirm and dismiss RPCs change status.	3.6	M	pgTAP: a student can't insert a confirmed fact directly; a mentor can confirm for their students; editing creates a new row that supersedes the old one
4.2	Memory proposer after an accepted Interview or Make, plus a "Bloom noticed... Save to my project?" card (keep, edit or dismiss).	4.1, 2.3	M	- At most 3 proposals per submission - A dismissed statement isn't proposed again (normalised text match) - Nothing becomes confirmed without a click

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
4.3	Brief compiler: deterministic project brief (fields, up to 12 confirmed facts, artifact summaries), cached and rebuilt when its inputs change.	4.1	S	• Snapshot test: same inputs give a byte-identical brief • Always under 1,200 tokens
4.4	Milestone map and gap check <code>lib/bloom/milestones.ts</code> , keyed by stage key (explore, choose, build, present).	3.6	S	• Unit test per stage: an empty project lists every expected artifact; a complete one lists none
4.5	Project-aware planning: <code>bloom_paths</code> gains <code>project_id</code> , <code>shape</code> and <code>target_artifact_type</code> ; the planner receives the brief and the gaps; "improve" mode when a current version exists; Decide writes decision facts directly.	4.3, 4.4, 1.7	M	• The worked example runs end to end • Asking for a business map when one exists produces an "improve your business map" path
4.6	Bloom home: project card (stage, expected and existing artifacts) and "Recommended next" (1–3).	4.4, 4.5	M	• Recommendations change after an artifact is accepted, without a page reload trick

Exit: each pilot student's project shows confirmed facts and artifacts, and the recommendation matches the stage gap.

LEARN

The propose-and-confirm pattern

Language models are good at noticing and phrasing, and unreliable as a source of truth. So AI output goes into a *proposed* state, and a person's click promotes it. Because changes create new rows that point to the old ones (*supersede*) instead of overwriting them, you can always answer "what did we believe in week 3, and why?". The same idea is behind the immutable artifact versions.

Why the brief is compiled, not generated

If an LLM summarised the project, the summary could drift or invent details, and you couldn't reproduce it. Compiling it from rows with code gives the same output for the same data, makes it testable, and costs nothing.

P5 Phase 5: Coach and mentors (MVP complete)

Goal Ask Bloom with memory; human judgement at milestones Flag `bloom_v2`, then all cohorts

Depends on 5.1–5.2: phases 0 and 2 only · 5.3–5.5: phase 4

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
5.1	Threads: a <code>bloom_messages</code> table (path, activity, role, text, flags); migrate <code>bloom_questions</code> to message pairs; keep a compatibility view.	0.3	M	- Migrated count is 2 × the number of questions - Old Q&A shows in the new thread UI - pgTAP visibility follows decision 0.1
5.2	Coach: assembled context plus the last 8 messages; suggested-action chips the student must click; safety topics flagged to the mentor and admin.	5.1, 2.3	M	- Eval: a follow-up such as "and what about the second one?" is answered correctly - Prompt size stays under about 6k tokens in a 40-message thread - Feedback never reveals a quiz answer key
5.3	Mentor check: activity type plus a <code>mentor_review</code> RPC (only <code>mentors_student</code> or admin); the problem statement becomes a milestone in the Choose stage.	4.4, 3.6	M	- pgTAP: another cohort's mentor gets "not allowed" - Waiting for a mentor doesn't block the student's other activities
5.4	Mentor queue: per group, lists awaiting review, "kept as is", "not reviewed", stuck students (2 revisions in a row, or no activity for 5+ days, via <code>loadLastActive</code>) and safety flags. Student view shows the brief, artifacts and facts.	5.3, 4.3	L	- Each item links to its source - Approving clears it from the queue - Works at tablet width
5.5	Parent view: current artifact versions and milestone outcomes; no drafts, proposals or AI assessments.	3.6, 5.3	S	pgTAP and a manual check against the visibility table in blueprint section 13

Exit (the MVP): two pilot cohorts complete a full stage with v2, then the flag is turned on for every cohort. Success measures: share of paths that produce an artifact, revision rate, mentor queue time, and AI cost per active student per week.

LEARN

Context windows and budgets

Each AI call can only "see" what's in its prompt. If you append every message forever, prompts grow, get slower and costlier, and models pay less attention to what matters. Fixed slices with budgets (project brief, this activity, recent messages) keep calls predictable. A rolling summary of older messages comes later (phase 2 of the blueprint).

Why mentor review is an RPC

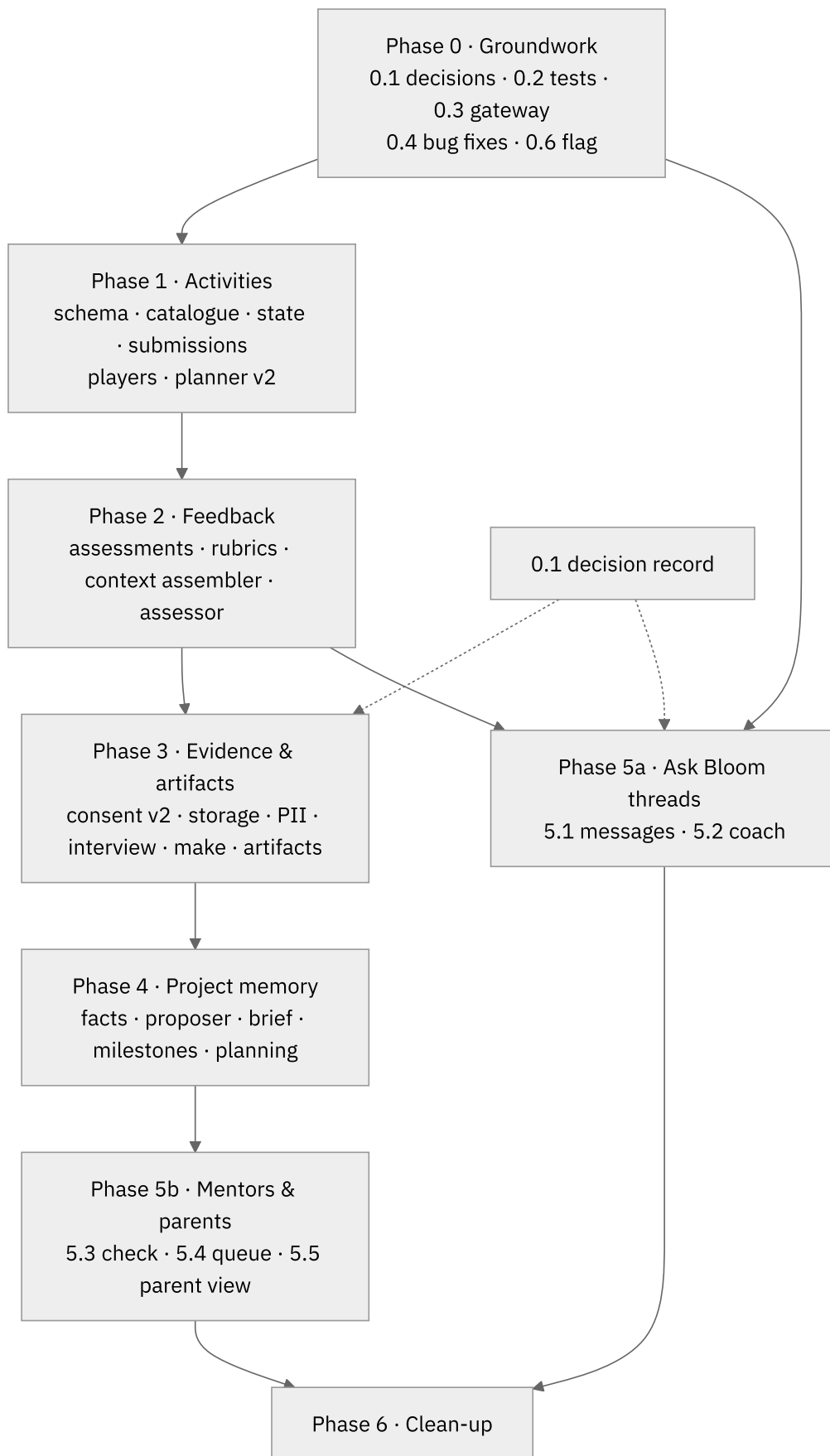
Students must never move their own work out of "awaiting mentor". Granting mentors general update rights on student tables would be too broad. A *security-definer* function runs with elevated rights but checks `mentors_student()` first and changes exactly one thing, like the existing `set_bloom_mentor_note` .

P6 Phase 6: Contract and clean up

Goal **remove transition scaffolding** Depends on **full rollout + 2 weeks stable**

ID	TASK	NEEDS	SIZE	ACCEPTANCE CRITERIA
6.1	Drop the <code>status ↔ state</code> sync trigger and the <code>status</code> column; drop <code>bloom_tasks.reflection</code> , the <code>bloom_questions</code> view and v1 planner code.	P5 rolled out	S	• No code references remain (grep plus type check) • All tests green • The data export still includes everything
6.2	Update <code>docs/architecture.md</code> , the as-built doc and the README; archive the blueprint as "implemented".	6.1	S	• The docs describe the running system

02 Dependencies



Critical path: 0.4 → 1.1 → 1.3 → 1.5 → 2.4 → 3.2 → 3.4 → 3.6 → 4.1 → 4.5 → 5.3 → 5.4. Threads (5.1–5.2) are off the critical path and suit a second developer once phase 2's context assembler exists.

Decision gates: consent v2 (3.1) can't ship without decision 0.1 and the new consent wording; thread visibility (5.1) follows the parent-visibility decision.

03 After the MVP (blueprint phase 2 and later)

- Mentor brief before each session (uses `sessions.starts_at` and the brief compiler)
- Try activity type (experiments); rank variant; link evidence
- Skill evidence and test-out on Learn
- Customer profile, prototype and pitch artifacts with mentor checkpoints at Build and Present
- Portfolio from final artifact versions (`projects.is_public` and `public_portfolio` consent)
- Rolling thread summaries; ranked recommendations
- Course activities producing artifacts, if decision 0.1 is revisited
- Organization-scoped admin rights, before a second organization joins
- Later: drawing canvas, voice, image moderation, Urdu, cross-cohort learning analytics

04 Glossary

TERM	MEANING HERE
RLS	Row-level security: Postgres policies that decide which rows a signed-in user can read or write.
pgTAP	The SQL test framework used in <code>supabase/tests</code> . Tests run inside a transaction that is rolled back.
Server action	A server function called directly from a form (<code>"use server"</code>); Bloom's only "API".
Security-definer RPC	A Postgres function that runs with its owner's rights and does its own permission check.
Feature flag	A row in <code>feature_flags</code> (global or per cohort), read by <code>isEnabled()</code> .
Expand / contract	Add new schema beside the old, migrate, then remove the old.
Eval	A labelled test set for judging AI output quality.
Artifact version	An immutable snapshot of a project artifact, created from an accepted submission.
Project fact	One confirmed statement about the project (insight, decision, assumption, open question).
Template / instance	Programme definition (programs, stages, activities) versus a running group (cohorts, memberships).